

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Environment setup

Step 1 Create a Single-Field Index

bash

mongosh

use university

```
db.createCollection("students")
```

```
db.students.insertMany([
```

```
{ name: "Alice Smith", age: 20, major: "Computer Science", gpa: 3.8, courses: ["CS101", "MATH201"] },
```

```
{ name: "Bob Johnson", age: 22, major: "Engineering", gpa: 3.5, courses: ["ENGR101", "PHYS201"] },
```

```
{ name: "Charlie Brown", age: 21, major: "Physics", gpa: 3.9, courses: ["PHYS101", "MATH201"] },
```

```
{ name: "David Lee", age: 19, major: "Computer Science", gpa: 3.7, courses: ["CS101", "MATH101"] },
```

```
{ name: "Eve Wilson", age: 23, major: "Biology", gpa: 3.6, courses: ["BIO101", "CHEM201"] },
```

```
{ name: "Alice Johnson", age: 20, major: "Engineering", gpa: 3.4, courses: ["ENGR101", "MATH101"] },
```

```
{ name: "Bob Smith", age: 21, major: "Physics", gpa: 3.8, courses: ["PHYS101", "CS101"] },
```

```
{ name: "Charlie Lee", age: 22, major: "Biology", gpa: 3.5, courses: ["BIO101", "PHYS201"] },
```

```
{ name: "David Brown", age: 19, major: "Computer Science", gpa: 4.0, courses: ["CS101", "ENGR101"] },
```

```
{ name: "Eve Johnson", age: 23, major: "Engineering", gpa: 3.9, courses: ["ENGR101", "CHEM201"] }
```

```
])
```

```
db.students.find().pretty()
```

```
db.students.createIndex( { age: 1 } )
```

```

test> use university
switched to db university
university> db.createCollection("students")
{ ok: 1 }
university> db.students.insertMany([
  { name: "Alice Smith", age: 20, major: "Computer Science", gpa: 3.8, courses: ["CS101", "MATH201"] },
  { name: "Bob Johnson", age: 22, major: "Engineering", gpa: 3.5, courses: ["ENGR101", "PHYS201"] },
  { name: "Charlie Brown", age: 21, major: "Physics", gpa: 3.9, courses: ["PHYS101", "MATH201"] },
  { name: "David Lee", age: 19, major: "Computer Science", gpa: 3.7, courses: ["CS101", "MATH101"] },
  { name: "Eve Wilson", age: 23, major: "Biology", gpa: 3.6, courses: ["BIOL101", "CHEM201"] },
  { name: "Alice Johnson", age: 20, major: "Engineering", gpa: 3.4, courses: ["ENGR101", "MATH101"] },
  { name: "Bob Smith", age: 21, major: "Physics", gpa: 3.8, courses: ["PHYS101", "CS101"] },
  { name: "Charlie Lee", age: 22, major: "Biology", gpa: 3.5, courses: ["BIOL101", "PHYS201"] },
  { name: "David Brown", age: 19, major: "Computer Science", gpa: 4.0, courses: ["CS101", "ENGR101"] },
  { name: "Eve Johnson", age: 23, major: "Engineering", gpa: 3.9, courses: ["ENGR101", "CHEM201"] }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69fc8fa53506aad43d3682d1'),
    '1': ObjectId('69fc8fa53506aad43d3682d2'),
    '2': ObjectId('69fc8fa53506aad43d3682d3'),
    '3': ObjectId('69fc8fa53506aad43d3682d4'),
    '4': ObjectId('69fc8fa53506aad43d3682d5'),
    '5': ObjectId('69fc8fa53506aad43d3682d6'),
    '6': ObjectId('69fc8fa53506aad43d3682d7'),
    '7': ObjectId('69fc8fa53506aad43d3682d8'),
    '8': ObjectId('69fc8fa53506aad43d3682d9'),
    '9': ObjectId('69fc8fa53506aad43d3682da')
  }
}
university> db.students.find().pretty()
[
  {
    _id: ObjectId('69fc8fa53506aad43d3682d1'),
    name: 'Alice Smith',
    age: 20,
    major: 'Computer Science',
    gpa: 3.8,
    courses: [ 'CS101', 'MATH201' ]
  },
  {
    _id: ObjectId('69fc8fa53506aad43d3682d2'),
    name: 'Bob Johnson',
    age: 22,
    major: 'Engineering',
    gpa: 3.5,
    courses: [ 'ENGR101', 'PHYS201' ]
  },
  {
    _id: ObjectId('69fc8fa53506aad43d3682d3'),
    name: 'Charlie Brown',
    age: 21,
    major: 'Physics',
    gpa: 3.9,
    courses: [ 'PHYS101', 'MATH201' ]
  },
  {
    _id: ObjectId('69fc8fa53506aad43d3682d4'),
    name: 'David Lee',
    age: 19,
    major: 'Computer Science',
    gpa: 3.7,
    courses: [ 'CS101', 'MATH101' ]
  },
  {
    _id: ObjectId('69fc8fa53506aad43d3682d5'),
    name: 'Eve Wilson',
    age: 23,
    major: 'Biology',
    gpa: 3.6,
    courses: [ 'BIOL101', 'CHEM201' ]
  },
  {
    _id: ObjectId('69fc8fa53506aad43d3682d6'),

```

```

{
  _id: ObjectId('69fc8fa53506aad43d3682d2'),
  name: 'Bob Johnson',
  age: 22,
  major: 'Engineering',
  gpa: 3.5,
  courses: [ 'ENGR101', 'PHYS201' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d3'),
  name: 'Charlie Brown',
  age: 21,
  major: 'Physics',
  gpa: 3.9,
  courses: [ 'PHYS101', 'MATH201' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d4'),
  name: 'David Lee',
  age: 19,
  major: 'Computer Science',
  gpa: 3.7,
  courses: [ 'CS101', 'MATH101' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d5'),
  name: 'Eve Wilson',
  age: 23,
  major: 'Biology',
  gpa: 3.6,
  courses: [ 'BIO101', 'CHEM201' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d6'),
  name: 'Alice Johnson',
  age: 20,
  major: 'Engineering',
  gpa: 3.4,
  courses: [ 'ENGR101', 'MATH101' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d7'),
  name: 'Bob Smith',
  age: 21,
  major: 'Physics',
  gpa: 3.8,
  courses: [ 'PHYS101', 'CS101' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d8'),
  name: 'Charlie Lee',
  age: 22,
  major: 'Biology',
  gpa: 3.5,
  courses: [ 'BIO101', 'PHYS201' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682d9'),
  name: 'David Brown',
  age: 19,
  major: 'Computer Science',
  gpa: 4,
  courses: [ 'CS101', 'ENGR101' ]
},
{
  _id: ObjectId('69fc8fa53506aad43d3682da'),
  name: 'Eve Johnson',
  age: 23,
  major: 'Engineering',
  gpa: 3.9,
  courses: [ 'ENGR101', 'CHEM201' ]
}
]
university>

```

```

university> db.students.createIndex( { age: 1 } )
age_1

```

Step 2 Analyze Query Execution Plan without Index

```

university> db.students.find( { age: 20 } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { age: { '$eq': 20 } },
    indexFilterSet: false,
    queryHash: '0637AF1A',
    planCacheShapeHash: '0637AF1A',
    planCacheKey: 'D54E87D9',
    optimizationTimeMillis: 8,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 13,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 11,

```

```

executionTimeMillis: 13,
totalKeysExamined: 2,
totalDocsExamined: 2,
executionStages: {
  isCached: false,
  stage: 'FETCH',
  nReturned: 2,
  executionTimeMillisEstimate: 11,
  works: 3,
  advanced: 2,
  needTime: 0,
  needYield: 0,
  saveState: 1,
  restoreState: 1,
  isEOF: 1,
  docsExamined: 2,
  alreadyHasObj: 0,
  inputStage: {
    stage: 'IXSCAN',
    nReturned: 2,
    executionTimeMillisEstimate: 11,
    works: 3,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    keyPattern: { age: 1 },
    indexName: 'age_1',
    isMultiKey: false,
    multiKeyPaths: { age: [] },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: { age: [ '[20, 20]' ] },
    keysExamined: 2,
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0
  }
},
queryShapeHash: 'ABCB17C9C0BEB9A543AA17AF6312D112544B44F4190E6E4CB0CCBD77380C8387',
command: { find: 'students', filter: { age: 20 }, '$db': 'university' },
serverInfo: {
  host: 'DESKTOP-4JDN53A',
  port: 27017,
  version: '8.2.7',
  gitVersion: '8f6785264a643d2eeb5e4046f7876a23e40e1a09'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1

```

Step 3 Analyze Query Execution Plan with Index

```
university> db.students.find( { age: 20 } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { age: { '$eq': 20 } },
    indexFilterSet: false,
    queryHash: '0637AF1A',
    planCacheShapeHash: '0637AF1A',
    planCacheKey: 'D54E87D9',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 0,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 2,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 2,
        executionTimeMillisEstimate: 0,
        works: 3,
        advanced: 2,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { age: 1 },
```

```

    executionTimeMillis: 0,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 2,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 2,
        executionTimeMillisEstimate: 0,
        works: 3,
        advanced: 2,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] },
        keysExamined: 2,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: 'ABC817C9C0BEB9A543AA17AF6312D112544B44F4190E6E4CB0CCBD77380C8387',
  command: { find: 'students', filter: { age: 20 }, '$db': 'university' },
  serverInfo: {
    host: 'DESKTOP-4JDN53A',
    port: 27017,
    version: '8.2.7',
    gitVersion: '8f6785264a643d2eeb5e4046f7876a23e40e1a09'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1

```

Step 4 Create a Compound Index

```
university> db.students.createIndex( { major: 1, gpa: -1 } )
major_1_gpa_-1
university> |
```

Step 5 Analyze Query Execution Plan with Compound Index

```
university> db.students.find( { major: "Computer Science" } ).sort( { gpa: -1 } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { major: { '$eq': 'Computer Science' } },
    indexFilterSet: false,
    queryHash: 'ECE62D24',
    planCacheShapeHash: 'ECE62D24',
    planCacheKey: '7D937067',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { major: 1, gpa: -1 },
        indexName: 'major_1_gpa_-1',
        isMultiKey: false,
        multiKeyPaths: { major: [], gpa: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          major: [ '['Computer Science', 'Computer Science']' ],
          gpa: [ '[MaxKey, MinKey]' ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 3,
    executionTimeMillis: 11,
    totalKeysExamined: 3,
    totalDocsExamined: 3,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 3,
      executionTimeMillisEstimate: 10,
      works: 4,
      advanced: 3,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 3,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 3,
        executionTimeMillisEstimate: 10,
        works: 4,
        advanced: 3,
        needTime: 0,
        needYield: 0,
```



```

        works: 4,
        advanced: 3,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        docsExamined: 3,
        alreadyHasObj: 0,
        inputStage: {
          stage: 'IXSCAN',
          nReturned: 3,
          executionTimeMillisEstimate: 10,
          works: 4,
          advanced: 3,
          needTime: 0,
          needYield: 0,
          saveState: 0,
          restoreState: 0,
          isEOF: 1,
          keyPattern: { major: 1, gpa: -1 },
          indexName: 'major_1_gpa_-1',
          isMultiKey: false,
          multiKeyPaths: { major: [], gpa: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            major: [ '['Computer Science', 'Computer Science'] ' ],
            gpa: [ '[MaxKey, MinKey]' ]
          },
          keysExamined: 3,
          seeks: 1,
          dupsTested: 0,
          dupsDropped: 0
        }
      }
    },
    queryShapeHash: '721EC999FC25193AFDB60F95F8C2413FDC31212425B0FE698305E16FACCB56B1',
    command: {
      find: 'students',
      filter: { major: 'Computer Science' },
      sort: { gpa: -1 },
      '$db': 'university'
    },
    serverInfo: {
      host: 'DESKTOP-4JDN53A',
      port: 27017,
      version: '8.2.7',
      gitVersion: '8f6785264a643d2eeb5e4046f7876a23e40e1a09'
    },
    serverParameters: {
      internalQueryFacetBufferSizeBytes: 104857600,
      internalQueryFacetMaxOutputDocSizeBytes: 104857600,
      internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
      internalDocumentSourceGroupMaxMemoryBytes: 104857600,
      internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
      internalQueryProhibitBlockingMergeOnMongoS: 0,
      internalQueryMaxAddToSetBytes: 104857600,
      internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
      internalQueryFrameworkControl: 'trySbeRestricted',
      internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
    },
    ok: 1
  }
}
university>

```

Step 6 Create a Multi-Key Index

```

university> db.students.createIndex( { courses: 1 } )
courses_1
university>

```

Step 7 Analyze Query Execution Plan with Multi-Key Index

```

university> db.students.find( { courses: "CS101" } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { courses: { '$eq': 'CS101' } },
    indexFilterSet: false,
    queryHash: '69CA0877',
    planCacheShapeHash: '69CA0877',
    planCacheKey: '285DD790',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { courses: 1 },
        indexName: 'courses_1',
        isMultiKey: true,
        multiKeyPaths: { courses: [ 'courses' ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { courses: [ '["CS101", "CS101"]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
    }
  }
}

```

```

    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 4,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 5,
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { courses: 1 },
        indexName: 'courses_1',
        isMultiKey: true,
        multiKeyPaths: { courses: [ 'courses' ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { courses: [ ['CS101', 'CS101'] ] },
        keysExamined: 4,
        seeks: 1,
        dupsTested: 4,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: '21E645F7646F91E9E554269E30096F16F5358F82CAE8ACC00825153891AF64EA',
  command: {
    find: 'students',
    filter: { courses: 'CS101' },
    '$db': 'university'
  },
  serverInfo: {
    host: 'DESKTOP-4JDN53A',
    port: 27017,
    version: '8.2.7',
    gitVersion: '8f6785264a643d2eeb5e4046f7876a23e40e1a09'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
university>

```

Step 8 Drop an Index

```
university> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { age: 1 }, name: 'age_1' },
  { v: 2, key: { major: 1, gpa: -1 }, name: 'major_1_gpa_-1' },
  { v: 2, key: { courses: 1 }, name: 'courses_1' }
]
university>
```

```
university> db.students.dropIndex("age_1")
{ nIndexesWas: 4, ok: 1 }
university>
```

Step 9 Drop All Indexes (Except the Default _id Index)

```
university> db.students.dropIndexes()
{
  nIndexesWas: 3,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
university>
```

Summary

The provided text highlights the critical role of indexes in optimizing MongoDB query performance. Without indexes, the database must perform a collection scan (COLLSCAN), reading every document to find matches, which becomes increasingly inefficient as datasets grow. By implementing indexes—such as single-field, compound, or multi-key indexes—MongoDB can utilize index scans (IXSCAN) to access relevant data directly. This transition significantly reduces the number of documents examined, leading to faster execution times for complex filtering, sorting, and array searches.

However, while indexes provide substantial benefits for read operations, they come with trade-offs. They require additional storage space and can introduce overhead that slows down insert and update operations, as the index must be updated alongside the data. Consequently, the text emphasizes that indexes should be applied strategically to balance high-speed query performance with resource efficiency and write speed.